

Understanding AWS Storage Options—From Zero to Hero

Getting Started

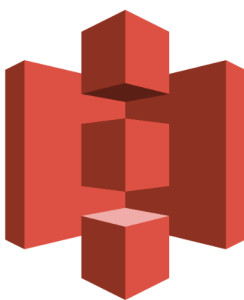
AWS storage services provide on-demand, scalable, and customizable storage. These storage services enable individual users and enterprises to deploy and access their data over the cloud. Meanwhile, the cloud provider manages the provision and maintenance of infrastructure for storage. Common uses of cloud storage are backups and recovery, including creating data lakes, performing analysis on large amounts of data, and providing access to data over the network.



In this lab, we'll learn about the three commonly used storage options in AWS: S3 Buckets, Elastic Block Store (EBS), and elastic file storage using the Elastic File System (EFS).

Get Introduced to Simple Storage Service

AWS Simple Service Storage (S3) stores data as objects in buckets. A **bucket** can store any data, including images, videos, files, and others. S3 is commonly used to host data for cloud-native applications, create data lake backups, save log files, and more. It's also used to host static websites.



The major features of S3 buckets include the following:

Structure of a bucket

A bucket has a flat structure instead of a hierarchical one, meaning all the data is stored on the same level instead of a hierarchy. Moreover, each bucket has a unique name with a unique identifier for each object. Therefore, we can access the bucket's content using its unique URL. This simplifies data retrieval.

The objects in a bucket contain both the data and metadata, such as the last modified date, size, content type, and others. A bucket can store virtually unlimited amounts of data, and the number of buckets an account can create depends on the AWS service quota for the account.

Security and access management

We can manage and customize access to the contents of a bucket using access-control lists (ACLs), block public access, and resource-based policies. **ACLs** configure read and write authorization for objects in the bucket.

To avoid accidental exposure of data, AWS provides a Block Public Access feature. It provides settings for bucket content, users, and the access points of a bucket. Block Public Access is enabled by default and is imposed irrespective of other access settings. We can change this setting and allow public access while

creating a bucket. However, disabling Block Public Access is not enough to allow public access. It needs to be coupled with a bucket policy to allow public access.

A **resource-based policy** bucket policy, for example, is similar to an IAM policy, which defines the allowed and denied actions and resources. Through a bucket policy, we can specify the users, objects, and read or write operations allowed.

Static website hosting

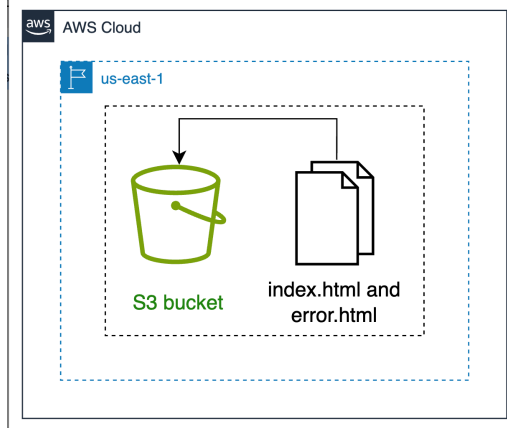
S3 buckets offer static website hosting. The contents of the website are uploaded to the bucket, and the bucket is configured to host the website. It generates a bucket URL to access the website.

When we enable the static website hosting property for an S3 bucket, we must upload an HTML document named `index.html`. If we have folders in our bucket, then every folder must contain a `index.html` file. S3 returns the default file of the accessed directory. Suppose we have a www.my-site.com endpoint, and accessing it will render the `index.html` file of the root directory of the S3 bucket. Now, if we have a folder named `movies` in the bucket, then www.my-site.com/movies/ will render the `index.html` page from the `movies` folder.

Now that we've understood the basic features of a bucket, in the next task, we'll learn to create a bucket and use some of its features.

Create an S3 Bucket and Upload Files

In this task, we'll create a simple S3 bucket and enable its static web hosting feature. The provisioned infrastructure at the end of this task is shown below:



Create a bucket

Follow the steps below to create an S3 bucket:

- Search for "S3" on the AWS Management Console and click "S3" from the search results. This takes us to the S3 dashboard.
- Click the "Create bucket" button.
- In the general configuration tab, make sure that the "AWS Region" is set to "US East (N. Virginia) us-east-1". If not, switch to the "N.Virgina" region using the drop-down menu on the top-right of the screen.
- Enter a name for the bucket. For this lab, `static-web-bucket` should be the first words of the bucket's name. Since the name of every bucket should be unique, append allowed characters with `static-web-bucket` to make the name globally unique. We can append the AWS account ID with the name of the bucket to ensure a distinct name for every bucket.
- Next, uncheck the "Block all public access" option. We'll later define a bucket policy to restrict public access. Acknowledge the public access warning by checking it.
- Keep default configurations for the rest of the settings.
- Scroll down to the end of the page and click the "Create bucket" button. We'll now be redirected to the S3 buckets page.

We've successfully created a bucket. Next, we'll upload files to it.

Upload files to the bucket

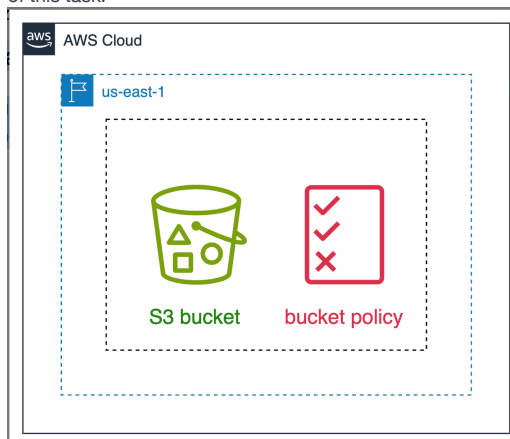
Follow the steps below to upload files to the bucket:

- Click the name of the bucket we've just created to open it.
- On the "Objects" tab, click the "Upload" button.
- Download the `index.html` and `error.html` files by clicking the download icon at the top-right corner of the widget below.
From the Github Link - <https://github.com/kamalsinghan-void/AWS-Cloud-Lab/tree/441f6ee58fa85eb97fe961259e3aea35398a6304/storage>
- In the "Files and folders" section, click the "Add files" button and select the `index.html` and `error.html` file from the system.
- Click the "Upload" button at the end of the page. It shows the "Upload: status" page. Once the file is uploaded, click the "Close" button to return to the bucket.

In this task, we've successfully created a bucket and uploaded files to it. Next, we'll enable static website hosting for the bucket.

Enable Static Website Hosting

In this task, we'll enable static website hosting for our bucket and add a bucket policy to our bucket. The illustration below depicts the provisioned infrastructure of this task:



Enable static website hosting

Let's enable the static website hosting property on our bucket using the following steps:

- Go to the S3 buckets page and click your bucket's name.
- Choose the "Properties" tab.
- Scroll down to the "Static website hosting" section at the bottom of the page, and click the "Edit" button.
- On the "Edit static website hosting" page, choose "Enable" under the "Static website hosting" section. It'll open up additional properties.
- Keep the hosting type "Host a static website" selected.
- In the "Index document" text field, give the file name you want to configure as a default file. When someone accesses your bucket website endpoint, this file will load. We'll add this file later on. Write `index.html` in the text field, and write `error.html` in the field for the error file. Remember that the file name is case-sensitive.
- Click the "Save changes" button.
- In the "Properties" tab, go to the "Static website hosting" section.
- We'll find a bucket website endpoint. Try to open it in a new tab of the browser. We'll get a `403 Forbidden` page because our bucket isn't publicly accessible for now. As we've discussed previously, besides disabling Block Public Access, buckets require a policy to allow public access.

Next, we'll add a bucket policy to allow access.

Configure bucket policy

Follow the steps below to add a bucket policy:

- Open the bucket and click the "Permissions" tab. Under this tab, we'll see the "Block public access (bucket settings)" and "Bucket policy" sections.
- Click the "Edit" button under the "Bucket policy" section. It redirects us to the "Edit bucket policy" page. Here, we have a policy editor to define bucket policy.
- Copy the code snippet given below and paste it into the editor.

This bucket policy allows read-only access to any resource available in our bucket. Your bucket's ARN is written above the policy editor. Copy and paste it in place of `<your-bucket-arn>` in the policy below:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "Statement1",
      "Principal": "*",
      "Effect": "Allow",
      "Action": [
        "s3:GetObject"
      ],
      "Resource": "<your-bucket-arn>/*"
    }
  ]
}
```

- Click the "Save changes" button, and the bucket policy will be updated.

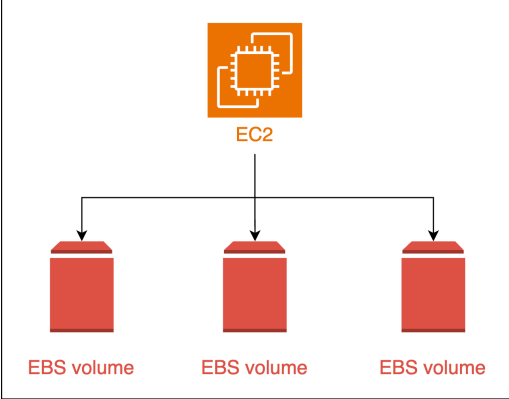
Now, go to the "Properties" tab. Scroll down and copy the endpoint URL in the "Static Website hosting" section. Paste this URL in the browser to view the `index.html` file.

In this section, we've created an S3 bucket and uploaded files to it. Also, we've enabled static website hosting and understood the bucket policy.

Get Introduced to EBS

AWS Elastic Block Storage (EBS) allows for storing data in the form of blocks. It's unformatted and raw disk storage that can be mounted to any EC2 instance. EBS volumes are commonly used for any task that would require a block device, such as a hard disk.

Every EC2 instance has an EBS volume attached to it when it's launched, which is deleted as soon as the instance is terminated. We can create additional volumes and attach them to EC2 instances. However, additional EBS volumes persist independently of the EC2 instance it's mounted to. Simply put, if we terminate the EC2 instance to which the additional volume has been mounted, the data will remain intact on the disk. We can access the data by mounting the volume to another EC2 instance. Also, we can attach multiple volumes to an EC2 instance.



EBS volumes are only accessible within the region they're created. To access them in another region, we need to create snapshots of the volume and replicate them. **Snapshots** are used to store backup for EBS volumes. They save the information required to restore an EBS volume. EBS offers a variety of volume types to cater to user requirements. The table below lists the types of volumes and their common use cases.

Types of EBS Volume

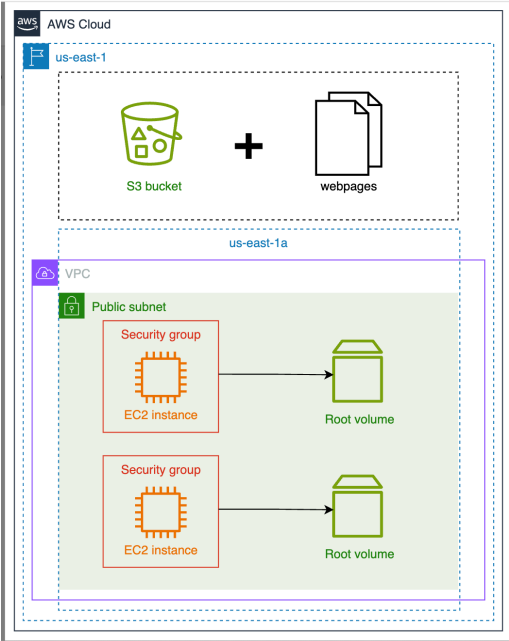
Type of Volume	Use Case
General Purpose SSD volumes (gp2, gp3)	Provide a balance between performance and price, and are the most commonly used
Provisioned IOPS SSD volumes (io1, io2)	Intensive input/ output workloads
Throughput Optimized HDD volumes (st1)	Low cost magnetic storage used for sequential storage
Cold HDD volumes (sc1)	For data accessed infrequently

In the next couple of tasks, we'll create an EBS volume and mount it to an EC2 instance. Let's start by launching an EC2 instance.

Launch an EC2 Instance

AWS Elastic Cloud Compute (EC2) allows us to create a scalable compute resource. Using EC2, we can deploy EC2 instances, which are essentially customizable virtual machines. To launch an EC2 instance, we'll require a security group. A **security group** is a virtual firewall that controls inbound and outbound traffic for EC2 instances. It defines rules, such as allowing SSH access, to protect instances from unauthorized access and ensure only the necessary traffic reaches our resources.

In this task, we'll create a security group and an EC2 instance. The infrastructure at the end of this lab will look similar to the one shown below:



The architecture diagram above shows a VPC and a public subnet. A **Virtual Private Cloud (VPC)** is an isolated virtual network within AWS that allows us to control the network space, including IP address ranges, subnets, and gateways. It provides the ability to define and control our own isolated network space, where we can launch AWS resources including EC2 instances.

where we can launch AWS resources, including EC2 instances.

Similarly, a **subnet** is a range of IP addresses within a VPC that divides the network into smaller segments for better management and distribution of resources across Availability Zones.

Create security group

Follow these steps to create a security group for our EC2 instances:

- Search for “EC2” in the search bar and click “EC2” from the results.
- From the sidebar, click “Security Groups.”
- On the “Security Groups” page, click the “Create security group” button.
- Name the security group as `ec2instances-sg`. Add the following text in the description: `Allows inbound SSH Traffic`.
- Click “Add rule” under “Inbound rules.” Select “SSH” from the drop-down menu under “Type.” Select “Anywhere-IPv4” from the drop-down menu under “Source.”
- Review the security group once to ensure it’s configured correctly. Click the “Create security group” button.

Launch an EC2 instance

Let’s launch an EC2 instance.

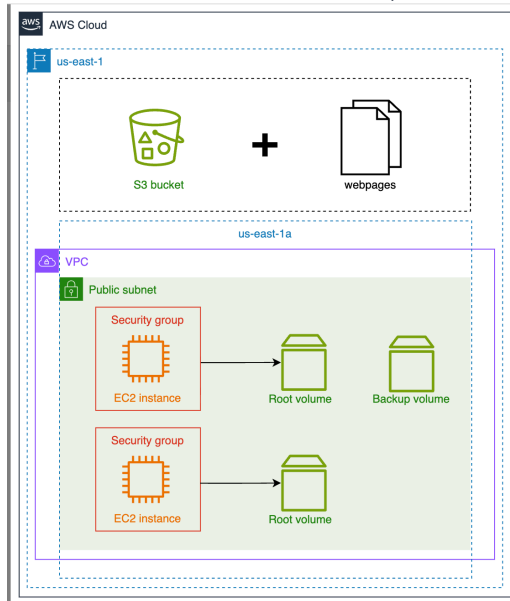
- In the EC2 dashboard, click the “Instances” option from the sidebar. This will lead us to the “Instances” page.
- Click the “Launch instances” button in the top-right corner.
- Set `instance-1` as the instance’s name.
- In the “Quick Start” menu under “Application and OS Images (Amazon Machine Image),” select “Amazon Linux.”
- From the drop-down menu, select “Amazon Linux 2023 kernel-6.1 AMI.”
- Keep the default “64-bit (x86)” option under the “Architecture” drop-down menu.
- Select `t3.micro` from the drop-down menu under “Instance type.”
- From the “Key pair name” drop-down menu, select the “Proceed without a key pair” option.
- Click the “Edit” button in the “Network settings” menu. Keep the default VPC. Select the default subnet in the `us-east-1a` Availability Zone from the drop-down menu under “Subnet.”
- Click “Select existing security group” and select `ec2instances-sg`.
- Choose `8GiB of gp3` storage under the “Configure storage” menu.
- We don’t need to adjust the advanced details for this lab, so we’ll keep the default settings under the “Advanced details” menu.
- Review the settings for the new instance before clicking the “Launch instance” button to launch the instance.

Click “View all instances” to return to the “Instances” page. Within a few minutes, the status of our newly launched instance should automatically change to “Running.”

Follow the same steps above to launch another instance named `instance-2` in `us-east-1a`. We’ll use this instance in the upcoming tasks. In the next task, we’ll create an EBS volume in the same Availability Zone.

Create an EBS Volume

In this task, we’ll create an EBS volume. The provisioned infrastructure is shown below:



Create an EBS volume

Follow the steps below to create an EBS volume:

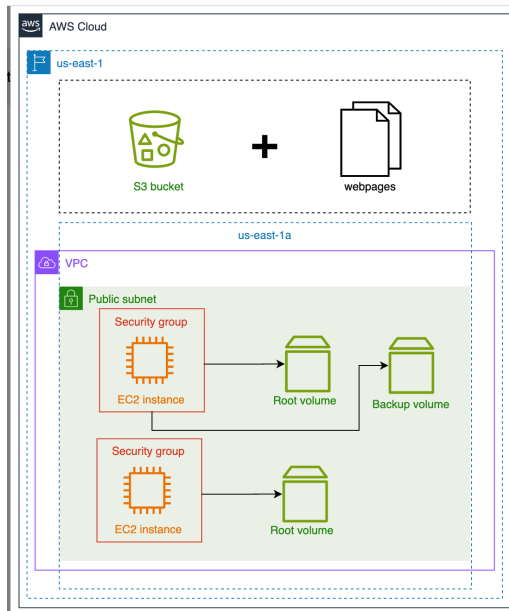
- On the EC2 dashboard, click the “Volumes” option under the “Elastic Block Store” section in the sidebar. This will lead us to the “Volumes” page.
- The “Volumes” page will display a list of EBS volumes. We should be able to see the volumes associated with `instance-1` and `instance-2`, the EC2 instances we created in the last task. These are the root volumes currently in use by our EC2 instances. As mentioned earlier, the root volume will be detached from the EC2 instance and deleted once we terminate our instance. We must create a new EBS volume and attach it to our EC2 instance if we want to retain the data in our storage. Let’s first rename the existing EBS volumes. Move the cursor near the “Name” cell of the existing volume. A small pencil will appear. Click the pencil to edit the name of the volume. Name one volume as `Root Volume 1` and the other as `Root Volume 2`.
- Click the “Create volume” button in the top-right corner.
- We’ve already discussed the different types of volumes. Here, we’ll use `gp3` to balance the cost and performance suitable for general-purpose workloads. Select the “General Purpose SSD (gp3)” volume type.
- Specify the size of the EBS volume to be `8 GiB`.
- Under the “Availability Zone” drop-down menu, select the same availability zone as your EC2 instance, which is `us-east-1a` in this case.
- We can add tags to the EBS volume to help with resource identification and management. For this task, we’ll skip adding tags.
- Review our EBS volume configuration and click the “Create volume” button.

After a few moments, our new EBS volume will appear in the “Volumes” list with the “Available” status, indicating it’s ready for use. Name this volume `Backup Volume`.

In the next task, we’ll attach and mount this volume to our EC2 instance.

Attach and Mount the EBS Volume

In this task, we’ll attach and mount the EBS volume on the EC2 instance and create a text file within the volume. The provisioned infrastructure is shown below:



Attach the EBS volume

Follow the steps below to attach the volume:

- Search “EC2” in the search bar of the AWS Management Console and click “Volumes” in the sidebar. Open the volumes dashboard to list the volumes.
- Make sure that the state of `Backup Volume` is “Available.” You can scroll right to see the state of the volume. Select the volume and click “Actions.”
- Select “Attach Volume” from the actions.
- Select `instance-1` from the list and click “Specify a custom device name...”. Enter the name `dev/sdf` as the “Device name” and click “Save” and “Next”. This is the name given by the EC2 instance to the volume. As mentioned in the note on the attached volume page, the device name changes from `dev/sdf` to `dev/xvdf`. Therefore, the volume attaches to the EC2 instance as `dev/xvdf`.

Note: On older-generation, Xen-based instances (for example, `t2.micro`), the Linux operating system renames this device from `/dev/sdf` to `/dev/xvdf`. In this case, the volume appears inside the instance as `/dev/xvdf`. Whereas newer-generation, Nitro-based instances (for example, `t3.micro`), the specified device name is not preserved by the operating system. Instead, the volume is exposed as an NVMe device (for example, `/dev/nvme1n1`).

- Click “Attach volume.” Notice that the state of the volume is changed to “In use.”

Now that the volume is attached, we can mount it to a directory in `instance-1`.

Instance Connect

To mount the volume, we’ll use Instance Connect. Follow the steps below to connect to the instance:

- On the EC2 dashboard, select `instance-1`.
- Click “Connect.”
- Keep the default option “Connect using EC2 Instance Connect” and click “Connect.” This will open the terminal.

Mount the volume

Follow the steps below to mount the volume:

- On the terminal, we can list the available block devices on an EC2 instance using the following command:

```
lsblk
```

The “Root Volume” associated with the instance has the device name `xvda` with 8 GiB storage capacity. The “Backup Volume” that we’ve created with the storage capacity of 8 GiB can be identified by the device name `xvdf`, as discussed above.

The storage device `xvdf` corresponding to our EBS volume doesn’t have an associated file system to organize and store files. In this step, we’ll create an `ext4` file system for this device.

- Create an `ext4` file system on the identified EBS volume using the following command:

```
sudo mkfs -t ext4 /dev/nvme1n1
```

Create a new directory to serve as the mount point for the EBS volume using the following command:

```
sudo mkdir /mnt/my-ebs-volume
```

Mount the EBS volume to the newly created mount point using the following command:

```
sudo mount /dev/nvme1n1 /mnt/my-ebs-volume
```

We can also edit the `/etc/fstab` file and configure the instance to mount the volume automatically on boot.

Add data to the new volume

- First switch to root user to avoid permission errors:

```
sudo su
```

Navigate to the mounted volume and create a text file with the content “This is a test file”.

```
cd /mnt/my-ebs-volume echo "This is a test file" > testfile.txt
```

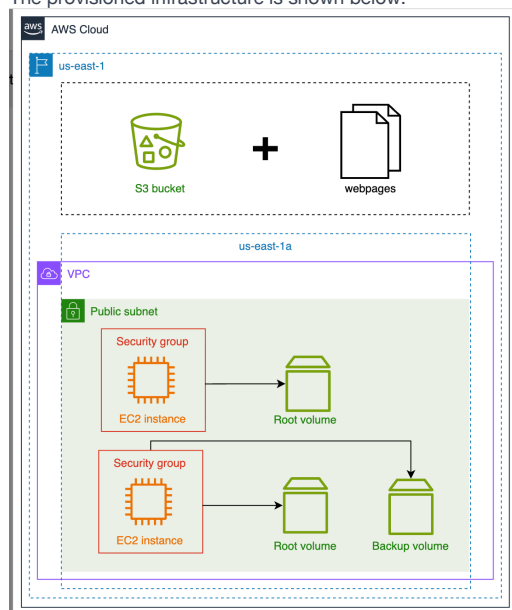
We’ve created a text file in the EBS volume. This file will remain stored on the volume even if we terminate instance-1. Therefore, we can access this file by mounting the volume to another EC2 instance. In the next task, we’ll read the contents of this file in instance-2.

Attach the Volume to Another EC2 Instance

We’ve already discussed that data persists in an EBS volume independent of the EC2 instance. Therefore, if we unmount a volume, detach it, and mount and attach it to another EC2 instance, the data persists in the volume.

In this part, we’ll unmount the volume, detach it from `instance-1`, and attach and mount the volume to `instance-2`, and then view the sample file.

The provisioned infrastructure is shown below:



Unmount the volume

Connect to the `instance-1` and run the following command in the terminal:

```
sudo umount -l /mnt/my-ebs-volume
```

Our volume is now unmounted.

Detach the volume

Follow the steps below to detach the volume:

- Open the EC2 dashboard and select “Volumes” from the sidebar.
- Select “Backup Volume” and choose “Actions.” Click “Force detach volume.”
- When prompted for confirmation, enter “detach” in the text box and click “Detach.”

Note: Before proceeding, make sure the volume is in the “Available” state. Refresh the page in case the state doesn't update.

Attach and mount the volume

Follow the steps we implemented in the previous task to attach and mount the volume.

Navigate to the `my-ebs-volume` directory using the command below:

```
cd /mnt/my-ebs-volume
```

Display the contents of the file using the following command:

```
cat testfile.txt
```

The file and its contents have persisted irrespective of the life of `instance-1`.

In this section, we've successfully created an EBS volume and mounted it to an EC2 instance. Also, we've understood data persistence by detaching it and attaching it to another EC2 instance.

Get Introduced to EFS

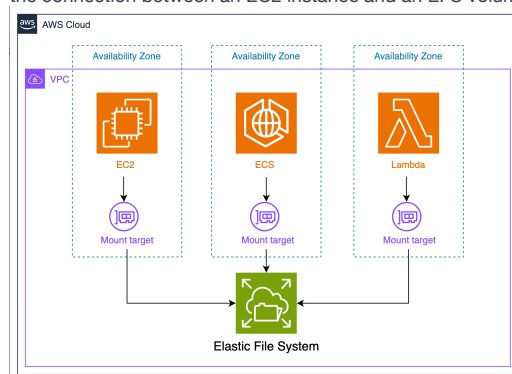
Amazon EFS is a file storage service. It provides the infrastructure for file storage, sparing users from developing and managing their file systems. It uses the Network File System protocol for communication with other resources.

EFS allows parallel access to the resources. EC2 instances, ECS instances, and Lambda functions can all access an EFS volume concurrently and simultaneously. It implements file locking and synchronous read-write operations to ensure data consistency. It can be scaled to petabytes and only charges for the storage capacity without a setup fee.

EFS is a serverless file storage system that provides event-driven services. It's also a regionally resilient service and offers multiple options for replication of data within an Availability Zone and among the Availability Zones.

How EFS works

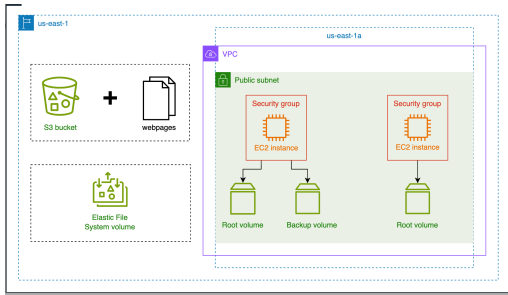
EFS volumes connect with compute instances using mount targets. A **mount target** provides an IP address for the NFS endpoint that can be used to mount to an EFS volume. Each compute instance in an Availability Zone connects to the EFS volume through a mount target. The illustration below shows the infrastructure for the connection between an EC2 instance and an EFS volume.



In this section, we'll create an EFS and mount two EC2 instances. One EC2 instance will be used to write a sample text file, while the other will be used to read the file.

Create an EFS Volume

In this section, we'll set up an EFS volume and create its mount target. The infrastructure at the end of the task will look similar to the architecture diagram below:



Create a security group

By default, all inbound traffic to a mount target is blocked, so we must explicitly create rules to allow traffic as per our requirements. To allow NFS traffic to an EC2 instance, our security group will allow NFS inbound traffic.

Follow these steps to create a security group for our EC2 instances:

- Search for “EC2” in the search bar and click “EC2” from the results.
- From the menu, click “Security Groups.”
- On the “Security Groups” page, click the “Create security group” button.
- Name the security group as **efs-sg**. Add the following text in the description: **Allows inbound NFS Traffic**.
- Select the default VPC under “VPC.”
- Click “Add rule” under “Inbound rules.” Select “NFS” from the drop-down menu under “Type.” Select “Anywhere-IPv4” from the drop-down menu under “Source.”
- Review the security group once to ensure that it’s configured correctly. Click the “Create security group” button.

Create a file system

Follow the steps below to create the file system:

- Search for “EFS” in the search bar and click “EFS” from the results.
- Click the “Create file system” button.
- Name the file system **clab-efs**.
- Select the default VPC from the “Virtual Private Cloud (VPC)” drop-down menu.
- Click the “Customize” button. We’ll be directed to the “File system settings” page. Here, we can configure the following settings:
 - **File system type:** The regional file system type allows us to save our file system data across multiple Availability Zones, providing more durability and availability. The “One Zone” storage class only stores the data within a single Availability Zone, which can help us reduce the cost of using the EFS volumes. Select “Regional” from the “File system type” options.
 - **Automatic backups:** By default, the automatic backups option is enabled. Since we’ll not be using this file system after the duration of this lab, we don’t need to create a backup for this EFS volume. Uncheck the “Enable automatic backups” option.
 - **Lifecycle management:** In this menu, we can choose after how many days our infrequently used files are moved into the Standard-Infrequent Access (IA) storage class. This can help cut down our cost significantly. By default, the transition to IA takes 30 days. For the purpose of this lab, set “Transition into Archive” to “None.”
 - **Encryption:** The data inside the file system is encrypted by default. It is recommended to keep the default setting for this option.
 - **Performance settings:** There are two ways to regulate the throughput limits of the file system.
 - **Bursting mode:** This scales the throughput based on the storage amount and is suitable for workloads with simple performance needs.
 - **Enhanced mode:** This offers greater flexibility and higher throughput for workloads with diverse performance requirements.
 - Select the enhanced mode for the purpose of this lab. Also, make sure “Elastic (Recommended)” is selected.
- Click the “Next” button. We’ll be directed to the “Network Access” page.
- Here, we can add mount targets for our file system. One mount target for each Availability Zone is already created with the default security group. Keep only two mount targets, one in **us-east-1a** and the other in **us-east-1b**. Moreover, remove the default security group and attach the security group **efs-sg**.
- Click the “Next” button. We’ll be directed to the “File system policy” page. Leave the policy editor empty and click the “Next” button.
- On the “Review and create” page, we can review the file system settings, network access, and file system policy. Scroll down to the end of the page and click the “Create” button.

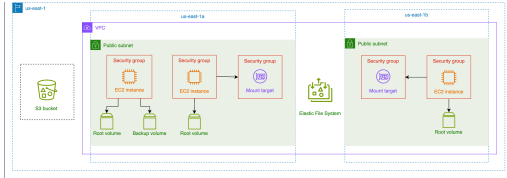
Once the file system is created, we need to copy the “File System ID” for the newly created file system and store it somewhere safe. We’ll need this ID later in the lab.

We’ve created an EFS volume. Next, we’ll create two EC2 instances to mount our file system to them.

Launch the EC2 Instance

In this task, we’ll create an instance. We’ll use the security group **ec2instances-sg** created in the previous section. After the completion of this task, the provisioned infrastructure will be similar to the one shown in the figure below:





Launch the EC2 instance

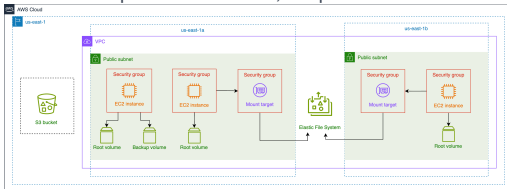
- Search for “EC2” in the search bar and select the “EC2” service from the search results. This will direct us to the EC2 dashboard.
- In the EC2 dashboard, click the “Instances” option in the left-hand menu. This will direct us to the “Instances” page.
- Click the “Launch instances” button in the top-right corner.
- Set the name of the instance to `instance-3`. We can skip adding additional tags for now and proceed to the next step.
- In the “Quick Start” menu under “Application and OS Images (Amazon Machine Image),” select “Amazon Linux”. Choose the “64-bit (x86)” option under the “Architecture” drop-down menu.
- Select `t3.micro` from the drop-down menu under “Instance type.”
- From the “Key pair name” drop-down menu, select the “Proceed without a key pair” option.
- Click the “Edit” button in the “Network Settings” menu.
- Keep the default VPC and select the same subnet as that of the mount target in `us-east-1b`.
- Click “Select existing security group” under the “Firewall (security groups)” menu. From the drop-down menu under “Common security groups,” select `ec2instances-sg` that we created earlier.
- Choose `8GiB` of `gp3` storage under the “Configure storage” menu.
- Review the settings for the new instance before clicking the “Launch instance” button to launch the instance.
- Click “View all instances” to return to the “Instances” page. Within a few minutes, the status of our newly created instance should automatically change to “Running.”

We have successfully created the EC2 instance. Next, we’ll mount the EFS volume to `instance-1` and `instance-3`.

Mount EFS Volume

In this task, we’ll mount our EFS volume to our EC2 instances. We’ll first install the Amazon EFS client on our instances. Then, we’ll create a mount point for our EFS volume and mount the EFS volume at that mount point.

After the completion of this task, the provisioned infrastructure will be similar to the one shown in the figure below:



Follow the steps below to mount the EFS volume on the EC2 instances:

- On the EC2 dashboard, click “Instances (running)” and then click the instance ID of our `instance-1` instance.
- On the “Instance summary” page, click the “Connect” button.
- Click the “Connect” button on the “Connect to instance” page. This will open up the terminal for our EC2 instance in a new tab.
- To install the `amazon-efs-utils` library, run the command given below in the terminal:

```
sudo yum install -y amazon-efs-utils
```

Here, we install `amazon-efs-utils`, which provides the Amazon EFS mount helper and utilities required to mount EFS volumes securely on the instance.

Next, we’ll create a directory to act as a mount point for our file system:

```
sudo mkdir /efsmount/
```

Mount the file system to `efsmount` using the command below. Replace `<fs-id>` with the ID of the file system.

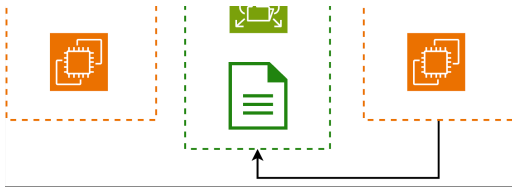
```
sudo mount -t efs <fs-id> /efsmount/
```

Repeat the steps above in this task for `instance-3` as well. Next, we’ll try to access this directory using both instances.

Create a File

In this task, we’ll create a file inside the EFS volume using one of the EC2 instances and access this file from the other EC2 instance.





Follow the steps below to show the shareability of files in the EFS volume across two different EC2 instances:

- Connect to `instance-1` as described in the previous task.
- Move to the `efsmount` directory using the following command:

```
cd /efsmount
```

Using the following command, switch to a superuser to escalate to root privileges:

```
sudo su
```

Create a text file using the following command:

```
echo "Creating a sample file from instance 1" >> file_from_1.txt
```

This will create a text file `file_from_1.txt` with the content "Creating a sample file from instance 1."

- Now connect to the `instance-3` as described in the "Attach and Mount the EBS Volume" task. Move into the `efsmount` directory using the command below:

```
cd /efsmount
```

- List the files in the directory using the following command:

```
ls
```

Once we run the `ls` command, we should be able to see the `file_from_1.txt` file, which was created using `instance-1`.

We've successfully mounted an EFS volume to two EC2 instances. Moreover, we've shown the shareability of the storage volume.

Clean Up

Let's end this lab by cleaning up all the resources we've created.

Terminate the instances

Follow the steps below to terminate the instances:

- Close the "EC2 Instance Connect" shell and return to the EC2 dashboard.
- Open up the sidebar and click the "Instances" option under the "Instances" section.
- Select `instance-1`, `instance-2`, and `instance-3` from the list.
- Click the "Instance state" drop-down button and then the "Terminate instance" option.
- Click the "Terminate" button.

Delete the EFS volume

Follow the steps below to delete the EFS volume:

- Search for "EFS" in the search bar and click "EFS." We'll be able to see a list of file systems.
- From the list, select the `clab-ef` file system and click the "Delete" button.
- Write the file system ID when prompted and click the "Confirm" button.

Delete the EBS volume

Navigate to the EC2 dashboard and follow the steps below to delete the volume:

- In the sidebar of the EC2 dashboard, click "Volumes."
- Select the `Backup Volume` volume by checking the box next to it.
- Click the "Actions" button at the top, then choose "Delete volume."
- Confirm the deletion in the confirmation dialog box.

Delete the security groups

We need to ensure that our instances and volumes are terminated and deleted. Otherwise, we won't be able to delete the security group.

- Open up the sidebar and click the "Security Groups" option under "Network & Security."
- Select the `efs-sg` and `ec2instances-sg` security groups from the list if they aren't already selected.
- Click the "Action" drop-down button and then the "Delete security groups" option.
- In the pop-up prompt, click the "Delete" button.

Delete the S3 bucket

Before we delete our S3 bucket, we must empty it. Follow the given steps to empty our S3 bucket:

- Head over to the “Amazon S3 > Buckets” page and search for your S3 bucket.
- Click the name of the S3 bucket.
- Select the S3 bucket we had created and click “Empty.” Enter “permanently delete” in the text field and click “Empty.”
- After the bucket is successfully emptied, we’ll see a success message. Click the “Exit” button.

Now that we’ve emptied the S3 bucket, we can delete the S3 bucket itself. Follow the given steps to delete the S3 bucket:

- Select the bucket from the bucket list.
- Click the “Delete” button.
- Enter the bucket name in the text box to confirm deletion and click the “Delete bucket” button.

By following the steps above, we’ll successfully clean up all the resources created during this lab.